YOUR FIRST LINUX KERNEL CONTRIBUTION

A practical guide to the Qualcomm Linux kernel source tree, regulator error handling, WSL setup, ARM64 cross-compilation, and patch validation

Hamsadurga S

Embedded Linux Engineer

What this guide covers

- What the panel driver does and what the proposed change is intended to improve.
- How the WSL Ubuntu environment was recovered and prepared for kernel development.
- How ARM64 cross-compilation works, including out-of-tree kernel configuration.
- The difference between compiling a driver object (.o) and linking a kernel module (.ko).
- How to interpret the modpost unresolved-symbol error and what the full build is expected to provide.
- How to use git diff, diff --check, and checkpatch.pl without confusing style validation with correctness.
- What remains to be verified before a patch can be committed or proposed upstream.

Current status at the time this guide was prepared: the modified driver object compiled successfully for ARM64; checkpatch reported zero errors and zero warnings; the standalone module link failed because the build tree lacked complete kernel symbol metadata. A full ARM64 kernel-and-modules build was the next planned validation step.

This is a learning and validation document, not confirmation that the patch is ready for upstream submission or that it fixes the original Waveshare touchscreen report.

1. Project context and the driver under investigation

The work has been taking place in a local clone of the public Qualcomm Linux kernel repository at **~/workspace/kernel**. The file being edited is **drivers/gpu/drm/panel/panel-focaltech-ota7290b.c**, a DRM panel driver for panels using the Focaltech OTA7290B controller.

The driver is a Linux DRM panel driver. Its prepare callback powers the panel and sends MIPI-DSI Display Command Set (DCS) commands to leave sleep mode and turn the display on. Its unprepare callback turns the display off, enters sleep mode, and disables the supplies.

Why this file was selected

The source review found that regulator_enable() failures in ota7290b_prepare() were logged but not consistently propagated. In the original implementation, execution could continue after a supply failed to enable. The DSI initialization result was returned, but partial power-up cleanup was not performed in the prepare failure path.

The proposed improvement is to stop on regulator failures, preserve the original error code, and roll back supplies that were successfully enabled earlier in the sequence.

Scope warning: the initial issue being investigated concerned a Waveshare 7-inch DSI touchscreen on Raspberry Pi 4B. The Focaltech OTA7290B driver is documented in this repository as an OTA7290B-based panel driver (the Kconfig help mentions a Waveshare 12.3-inch DSI TOUCH-A example). Do not claim that this patch fixes the 7-inch issue unless the actual controller, compatible string, panel wiring, and failure mechanism are shown to match.

Important terms

Term	Meaning in this work
DRM	Direct Rendering Manager, the Linux graphics subsystem.
Panel driver	A driver that manages a display panel's power, reset, and display command sequence.
MIPI-DSI	Display serial interface used to connect the display host and panel.
DCS	Display Command Set commands such as exit-sleep-mode and display-on.
Regulator	Linux framework abstraction for a controllable power supply rail.
GPIO descriptor	The descriptor-based GPIO API used here for panel reset.
prepare()	DRM panel callback that powers and initializes the panel.
unprepare()	DRM panel callback that shuts down the panel and its supplies.

2. Understanding the proposed error-handling change

The panel uses optional regulator handles named vcc, vdd, and avdd, plus an optional reset GPIO. The existing sequence enables VCC, performs a reset sequence, enables VDD, performs another reset operation, enables AVDD, then sends DSI commands.

Intended failure handling

Failure point	Intended action
VCC enable fails	Return the regulator error immediately. No earlier regulator in this sequence has been enabled.
VDD enable fails	Disable VCC if it was present and enabled.
AVDD enable fails	Restore the reset state, then disable VDD and VCC.
DSI initialization fails	Restore reset, disable AVDD, then disable VDD and VCC.

The labels in the current diff implement a reverse-order rollback pattern. Reverse-order cleanup is common when resources are acquired in sequence: if A is enabled, then B, then C, a failure after C should normally release C, then B, then A.

Why prepare() must clean up after itself

The DRM panel core calls the panel driver's prepare callback and checks its return value. A failed prepare does not imply that the core will automatically unwind every regulator enabled by the driver's callback. Therefore the callback must have a well-defined failure path that releases resources acquired before the failure.

Subtle points to verify before submission

- **GPIO polarity:** gpiod_set_value_cansleep() uses logical values. Whether logical 1 asserts reset depends on the GPIO descriptor polarity supplied by firmware/device tree. The existing unprepare() sequence uses logical 1, but the binding and board description should still be checked.
- **Power sequencing:** confirm from the panel datasheet or board design that asserting reset before disabling the rails is correct, and that the delays are adequate.
- **Optional regulators:** verify how each optional regulator is acquired and that NULL is the expected absent-supply representation in this driver.
- **Disable return values:** regulator_disable() can return an error. Review local subsystem conventions and decide whether cleanup errors should be logged; do not overwrite the original prepare failure without a deliberate reason.
- **Partial DSI command failure:** the DSI multi-context accumulates an error. Check whether a DSI shutdown command is appropriate after a partial initialization failure; do not assume it will succeed after the transport has already failed.

The exact final diff at the last review changed only ota7290b_prepare(). It added immediate VCC error return, rollback labels for VDD and AVDD failures, and rollback on DSI initialization failure. It also used usleep_range(5000, 6000) for the newly added 5 ms reset cleanup delays. The existing unprepare() msleep(5) was restored to avoid an unrelated change.

3. WSL and Ubuntu development environment

The development machine runs Windows with Ubuntu 26.04 under WSL 2. The Linux shell prompt showed the user as durga, with the source tree at ~/workspace/kernel.

Why sudo initially failed

The package installation failed because the Ubuntu user's Linux password was not accepted. The Windows account password or Windows Hello PIN is not necessarily the same as the Ubuntu Linux password. Password entry in a Linux terminal is intentionally invisible: no characters or asterisks appear while typing.

The WSL distribution was checked from Windows PowerShell using:

```
wsl --list --verbose
```

The output identified Ubuntu-26.04, WSL version 2. The command wsl is a Windows-side command; typing it inside the Ubuntu shell produces 'command not found'. Similarly, apt is a Linux package manager and should be run inside Ubuntu, not PowerShell.

The password recovery path used a root shell launched from elevated Windows PowerShell:

```
wsl --user root
```

Then, inside the root Ubuntu shell, the Linux account password was reset:

```
passwd durga
```

After exiting root, sudo was available again in the regular Ubuntu session. No Ubuntu reinstall or repository deletion was required.

Build dependencies installed

```
sudo apt update

sudo apt install -y \
    build-essential \
    gcc-aarch64-linux-gnu \
    binutils-aarch64-linux-gnu \
    bc bison flex \
    libssl-dev libelf-dev libncurses-dev \
    dwarves rsync
```

These packages provide the host build tools, ARM64 cross-compiler and binutils, kernel configuration/build dependencies, and supporting utilities. The exact dependency set can vary by kernel version and enabled configuration.

Resource snapshot

Resource	Observed value	Build implication
CPU	4 logical CPUs	Use modest parallelism.
RAM	5.7 GiB total; 4.2 GiB available at the time	Avoid very high -j values.
Swap	2 GiB configured	May help under pressure but is slower than RAM.
Workspace filesystem	About 944 GiB free	Disk space was not a limiting factor in the snapshot.

A two-job build (-j2) was recommended as a conservative starting point for this memory size. Actual build duration depends on CPU speed, kernel configuration, cache state, and source-tree size. The earlier 30–90 minute estimate is only a rough planning range, not a guarantee.

4. ARM64 cross-compilation and out-of-tree build

The WSL machine reports uname -m as x86_64. That describes the architecture of the Linux environment running inside WSL. The target being built is ARM64, so the build uses an ARM64 cross-compiler.

What ARCH and CROSS_COMPILE mean

Build variable	Purpose
ARCH=arm64	Selects the Linux kernel architecture and ARM64-specific build rules.
CROSS_COMPILE=aarch64-linux-gnu-	Prefixes target compiler/binutils commands, such as aarch64-linux-gnu-gcc.
O=~/workspace/kernel-build	Places generated build output and .config outside the source tree.

Out-of-tree builds keep generated files such as .config, generated headers, object files, and module outputs separate from the Git source checkout. This makes it easier to inspect the source diff and avoid accidentally committing generated build products.

The source and build directories

```
Source tree:
    ~/workspace/kernel/

Separate build output:
    ~/workspace/kernel-build/
```

Kernel configuration

The repository contained arch/arm64/configs/defconfig. The initial configuration command was:

```
cd ~/workspace/kernel

make O=~/workspace/kernel-build \
     ARCH=arm64 \
     CROSS_COMPILE=aarch64-linux-gnu- \
     defconfig
```

This successfully created ~/workspace/kernel-build/.config. The .config file is the resolved set of kernel configuration symbols used by Kbuild.

Enabling the panel driver

The Kconfig entry was found in drivers/gpu/drm/panel/Kconfig:

```
config DRM_PANEL_FOCALTECH_OTA7290B
    tristate "Focaltech OTA7290B"
    depends on DRM_MIPI_DSI
    depends on I2C
    depends on BACKLIGHT_CLASS_DEVICE
    select DRM_KMS_HELPER
```

The Makefile maps that configuration symbol to the object:

```
obj-$(CONFIG_DRM_PANEL_FOCALTECH_OTA7290B) += panel-focaltech-ota7290b.o
```

The driver was selected as a module and dependencies were resolved:

```
./scripts/config \
    --file ~/workspace/kernel-build/.config \
    -m DRM_PANEL_FOCALTECH_OTA7290B

make O=~/workspace/kernel-build \
     ARCH=arm64 \
     CROSS_COMPILE=aarch64-linux-gnu- \
     olddefconfig

grep CONFIG_DRM_PANEL_FOCALTECH_OTA7290B \
    ~/workspace/kernel-build/.config
```

The final grep output was CONFIG_DRM_PANEL_FOCALTECH_OTA7290B=m. In Kconfig tristate notation, m means build as a loadable module; y means built into the kernel; n means disabled.

5. Object compilation versus module linking

Stage A: compile the driver object

The command used was:

```
make O=~/workspace/kernel-build \
     ARCH=arm64 \
     CROSS_COMPILE=aarch64-linux-gnu- \
     drivers/gpu/drm/panel/panel-focaltech-ota7290b.o
```

The build ended with:

```
CC [M] drivers/gpu/drm/panel/panel-focaltech-ota7290b.o
```

This is a successful compile of the modified C source into an ARM64 relocatable object. It confirms that the compiler accepted the source and that the headers and declarations needed for this translation unit were available in the configured build.

It does **not** prove that the object can be linked into a loadable kernel module, that all kernel symbols are available, or that the driver works on a physical panel.

Stage B: why the directory build did not create the .ko

A directory-level request built configured objects in that directory, but the output did not include the expected panel-focaltech-ota7290b.ko. The next step explicitly requested the .ko target.

Stage C: module linking and modpost

The explicit .ko request reached module post-processing and failed with messages including:

```
WARNING: vmlinux.o is missing.
ERROR: modpost: symbol 'regulator_enable' undefined!
ERROR: modpost: symbol 'gpiod_set_value_cansleep' undefined!
ERROR: modpost: symbol 'mipi_dsi_dcs_exit_sleep_mode_multi' undefined!
```

The key diagnosis is the missing complete kernel symbol metadata. The build had a configuration and the individual object, but had not completed the full kernel build that generates the symbol information modpost needs. This is a build completeness problem; these messages alone do not establish that the APIs are absent from the source tree.

What modpost does

modpost is a kernel build step that examines module objects and checks their references against the kernel's exported symbol information. For a module build, Kbuild needs the appropriate kernel symbol/version metadata, commonly represented through Module.symvers and related full-build outputs. Without that information, valid kernel references can be reported as unresolved.

Planned Stage D: full kernel and modules build

The next planned command, not yet confirmed complete in the conversation, is:

```
cd ~/workspace/kernel

make -j2 \
    O=~/workspace/kernel-build \
    ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- \
    Image modules
```

This asks Kbuild to build the ARM64 kernel image and configured modules. It is expected to generate the kernel build metadata needed for modpost. If the build completes successfully, the module can then be checked at:

```
ls -lh ~/workspace/kernel-build/drivers/gpu/drm/panel/panel-focaltech-ota7290b.ko
```

> **Do not treat this as already successful:** the full Image + modules build was the next action at the time of writing. It has not yet been reported as completed.

After a successful full build

```
make -j2 \
```

```
O=~/workspace/kernel-build \
ARCH=arm64 \
CROSS_COMPILE=aarch64-linux-gnu- \
drivers/gpu/drm/panel/panel-focaltech-ota7290b.ko
```

If Kbuild reports a new error, preserve the complete error output. Do not suppress unresolved-symbol errors with KBUILD_MODPOST_WARN=1 merely to produce a .ko; that would hide the validation problem rather than resolve it.

6. Patch review and kernel style checks

Git commands were used to confirm exactly what changed. The source modification was limited to drivers/gpu/drm/panel/panel-focaltech-ota7290b.c.

Useful commands

```
# From the repository root
git status --short
git diff --stat
git --no-pager diff -- drivers/gpu/drm/panel/panel-focaltech-ota7290b.c
git diff --check

git diff -- drivers/gpu/drm/panel/panel-focaltech-ota7290b.c | \
    scripts/checkpatch.pl --strict -
```

The first diff inspection was accidentally run from inside drivers/gpu/drm/panel using a repository-root-relative path. From inside that directory, the correct path is just panel-focaltech-ota7290b.c. From the repository root, use the full relative path.

What the checks mean

Check	What it tells you	What it does not tell you
git status --short	Which tracked/untracked files are modified.	Whether the changes are correct.
git diff --stat	A summary of changed files and line counts.	Whether the patch is appropriate.
git diff	The exact source changes relative to the index/HEAD.	Whether the hardware sequence is valid.
git diff --check	Detects whitespace errors such as trailing whitespace and blank lines at EOF.	Kernel coding style or logic issues.
checkpatch.pl --strict	Reports many Linux kernel patch style issues.	Functional correctness, design quality, or maintainability.
Object compile	The translation unit compiles for the selected target/configuration.	Module linking or runtime behaviour.
Module build	Kbuild links and post-processes the module against kernel build metadata.	Hardware validation.

The latest reported checkpatch result was: total: 0 errors, 0 warnings, 0 checks, 79 lines checked. This means no obvious style problems were reported for the patch. It is not an upstream approval.

Why the patch went through several edits

During editing, extra blank lines, inconsistent label indentation, and accidental double semicolons appeared. The strict checkpatch run exposed the short-delay msleep warnings and later the double semicolons. The final reviewed diff removed the unrelated change to unprepare() and kept the new usleep_range(5000, 6000) calls only in the new prepare() cleanup paths.

The final diff summary reported one file changed, 45 insertions and 9 deletions. The final strict checkpatch output reported zero errors and zero warnings.

7. Remaining technical validation before a commit

The current patch is a proposed error-handling improvement. It should not be committed solely because checkpatch passes and the object compiles. Complete the following review.

A. Inspect the final source and patch

```
cd ~/workspace/kernel
git status --short
git diff --check
git --no-pager diff -- drivers/gpu/drm/panel/panel-focaltech-ota7290b.c
```

B. Re-read surrounding code

Read the complete prepare() and unprepare() functions, the regulator acquisition code in probe(), the GPIO descriptor acquisition and polarity in the device tree/binding, and the driver documentation. Verify that the error labels correspond exactly to resources successfully enabled on each path.

C. Review cleanup semantics

- For every regulator_enable() success, identify the matching regulator_disable() on both normal shutdown and every prepare() failure path.
- For a regulator_enable() failure, do not disable that same regulator unless the API contract or driver design specifically requires it; the enable call did not succeed.
- Ensure cleanup does not disable a supply that was never enabled on the path being taken.
- Preserve the original failure code in ret. If cleanup itself fails, decide whether to log that secondary failure while retaining the original cause.
- Verify reset assertion/deassertion semantics against the active-low/active-high GPIO flags and panel datasheet.
- Verify the 5–6 ms delay requirement against the panel reset/power timing specification and nearby driver conventions.

D. Build validation

1. Confirm the complete Image + modules build finishes without errors.
2. Confirm the .ko file exists in the out-of-tree build directory.
3. Review the complete build log for warnings that concern this driver.
4. If practical, build with the exact target kernel configuration/branch used by the intended Qualcomm platform, not only generic ARM64 defconfig.

E. Functional validation

A successful cross-build does not replace hardware testing. Ideally test on a board with the matching OTA7290B panel and verify normal boot/display operation, regulator enable failures where practical, DSI initialization failure behaviour, repeated prepare/unprepare cycles, and absence of regulator reference-count leaks. If hardware is unavailable, state that clearly in the patch discussion.

F. Confirm patch relevance

The selected driver is for the OTA7290B controller. The original Waveshare 7-inch Raspberry Pi 4B report must be checked against its actual panel controller and device-tree compatible. If it uses a different controller, this patch is not a fix for that report. It may still be a separate valid cleanup bug, but it needs its own rationale and evidence.

G. Confirm contribution target

Before creating a commit or pull request, inspect the repository's CONTRIBUTING documentation, MAINTAINERS entries, branch policy, sign-off requirements, and expected submission route. Do not assume that main is the correct target branch for a Qualcomm downstream kernel. If you do not have write access to the upstream repository, the normal GitHub workflow is to fork the repository, push a topic branch to your fork, and open a pull request against the

correct upstream branch.

A Signed-off-by line is commonly required for Linux kernel contributions. Only add it if you agree to the Developer Certificate of Origin and the repository's contribution requirements. The typical command, once all review is complete, is:

```
git commit -s -v
```

Do not run this yet. First establish that the patch is technically sound and relevant.

8. End-to-end command reference

This section collects the principal commands used in the workflow. Run them from the stated shell and directory; do not paste Linux commands into Windows PowerShell unless explicitly marked as PowerShell.

Windows PowerShell

```
# Check WSL distributions and versions
wsl --list --verbose

# Start Ubuntu as root for account recovery, if required
wsl --user root
```

Ubuntu / WSL

```
# Install build tools (after sudo access works)
sudo apt update
sudo apt install -y build-essential gcc-aarch64-linux-gnu \
    binutils-aarch64-linux-gnu bc bison flex libssl-dev \
    libelf-dev libncurses-dev dwarves rsync

# Configure ARM64 out-of-tree build
cd ~/workspace/kernel
mkdir -p ~/workspace/kernel-build
make O=~/workspace/kernel-build ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- defconfig

# Enable the driver as a module
./scripts/config --file ~/workspace/kernel-build/.config \
    -m DRM_PANEL_FOCALTECH_OTA7290B
make O=~/workspace/kernel-build ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- olddefconfig

# Verify
grep CONFIG_DRM_PANEL_FOCALTECH_OTA7290B \
    ~/workspace/kernel-build/.config

# Compile only the driver object
make O=~/workspace/kernel-build ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- \
    drivers/gpu/drm/panel/panel-focaltech-ota7290b.o

# Full kernel and modules build (planned next validation)
make -j2 O=~/workspace/kernel-build ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- Image modules

# Verify module after successful full build
ls -lh ~/workspace/kernel-build/drivers/gpu/drm/panel/\
panel-focaltech-ota7290b.ko
```

Git validation

```
cd ~/workspace/kernel
git status --short
git diff --stat
git diff --check
git --no-pager diff -- drivers/gpu/drm/panel/panel-focaltech-ota7290b.c
git diff -- drivers/gpu/drm/panel/panel-focaltech-ota7290b.c | \
    scripts/checkpatch.pl --strict -
```

Vocabulary recap

Word	Short definition
Host	The machine/environment running the build (here, x86_64 WSL Ubuntu).
Target	The architecture for which the output is built (here, ARM64).
Cross-compiler	Compiler running on the host that emits code for the target architecture.
Kbuild	The Linux kernel's Make-based build system.
Kconfig	The kernel configuration system defining options and dependencies.
defconfig	A baseline configuration for an architecture or platform.
olddefconfig	Resolves configuration dependencies and defaults without interactive prompts.
Object (.o)	Compiled object code, not yet necessarily a complete loadable module.
Module (.ko)	A linked kernel module.
modpost	Kernel build stage that checks module symbols and metadata.
Module.symvers	Build-generated information about exported symbols and symbol versions.
Out-of-tree build	Build output stored separately from the source checkout.

9. Current project status and immediate next action

Milestone	Status based on the conversation
WSL Ubuntu 26.04 / WSL 2 identified	Done
Ubuntu password reset and sudo access	Done
Kernel build dependencies installed	Installation was initiated; compiler was available for the successful object build.
ARM64 defconfig in separate build directory	Done
Focaltech OTA7290B selected as module	Done; CONFIG_DRM_PANEL_FOCALTECH_OTA7290B=m
Driver object compilation	Done; CC [M] object completed successfully
git diff --check	Passed
checkpatch.pl --strict	Passed; zero errors and zero warnings
Standalone .ko link	Failed at modpost due to missing full kernel symbol metadata
Full Image + modules build	Next planned step; completion not yet reported
Hardware validation	Not yet performed
Commit / upstream submission	Not yet performed

The immediate next command

```
cd ~/workspace/kernel

make -j2 \
    O=~/workspace/kernel-build \
    ARCH=arm64 \
    CROSS_COMPILE=aarch64-linux-gnu- \
    Image modules
```

Let the build finish and capture its final output. If it succeeds, verify that panel-focaltech-ota7290b.ko exists. If it fails, share the first meaningful error and the final section of the log; the first error is often more useful than the cascade of later failures.

Final takeaway: you have already completed several authentic kernel-development steps: inspecting Kconfig/Makefile integration, editing a DRM panel driver, reviewing a Git patch, running checkpatch, configuring an ARM64 cross-build, and compiling the modified object. The remaining work is to complete module linking, validate the rollback logic against the hardware/software contracts, confirm the patch's relevance and target branch, and only then decide whether to submit it.